# “Resume Search and Candidate Screening System”

*Project Report submitted for the partial fulfilment of grade for the course*

**Data Structures and Algorithms -3 (25CS2103E)**

**in**

**DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**

by


Ishaan Kulkarni — 2520030267

Mohammad Faizaan — 2520030359

Abhiram Koppisetty — 2520030357

Team Number: 6

Section: 9


Under the Guidance of


**Dr. S. Vinay Kumar**
**Associate Professor**
**Department of Computer Science and Engineering**






**Koneru Lakshmaiah Education Foundation**
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in


**2026-27**

# Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

# DECLARATION

We declare that the project report entitled “Resume Search and Candidate Screening System” records the work carried out by Ishaan Kulkarni (2520030267), Mohammad Faizaan (2520030359), and Abhiram Koppisetty (2520030357), Team 6, Section 9, for the course Data Structures and Algorithms–3 (25CS2103E). The report describes the design and implementation of our academic project. The included candidate records are synthetic and are used only for demonstration.

**Ishaan Kulkarni — 2520030267**

**Mohammad Faizaan — 2520030359**

**Abhiram Koppisetty — 2520030357**

# CERTIFICATE

This is to certify that the project report entitled “Resume Search and Candidate Screening System” is a record of the work carried out by Ishaan Kulkarni (2520030267), Mohammad Faizaan (2520030359), and Abhiram Koppisetty (2520030357), Team 6, Section 9, in partial fulfilment of the requirements for the course Data Structures and Algorithms–3 (25CS2103E), during the academic year 2026–27.

**Signature of the Guide**

Dr. S. Vinay Kumar

Associate Professor

Department of Computer Science and Engineering

**Signature of the HOD**

Dr. P. Venkateshwara Rao

Associate Professor and Head of the Department

**Signature of the External Examiner**

# ACKNOWLEDGEMENT

We would like to thank our project guide, Dr. S. Vinay Kumar, Associate Professor, Department of Computer Science and Engineering, for his guidance and feedback during the development of this project. His suggestions helped us organize the implementation into separate modules and focus on the algorithmic requirements of the course.

We also thank Dr. P. Venkateshwara Rao, Associate Professor and Head of the Department of Computer Science and Engineering, for the academic support provided by the department. We are grateful to the university for giving us the opportunity to apply data structures and algorithms to a practical problem.

Finally, we thank our teammates for their contributions to the Java screening engine, the synthetic resume dataset, and the website interface.

Ishaan Kulkarni — 2520030267

Mohammad Faizaan — 2520030359

Abhiram Koppisetty — 2520030357

# ABSTRACT

Resume screening is an important part of recruitment because applications need to be compared against the requirements of a role. Reviewing every profile manually can be repetitive, especially when the dataset grows. The Resume Search and Candidate Screening System provides a local prototype that searches a collection of candidate profiles using skills, minimum experience, and preferred location, then ranks the results using a weighted fit score.

The Java console application loads candidate records from a text file and represents each record as a Candidate object. The repository maintains a collection of profiles and a skill index, while the screening service and ranking module handle search criteria and score calculation. A separate website built with HTML, CSS, and JavaScript provides Candidate Search, Resume Database, and Matching Insights views. The ranking model uses 50% for skills, 30% for experience, and 20% for location.

The project also contains runnable demonstrations of selected data structures and algorithms, including string matching, dynamic programming, maximum flow, approximation, randomized sorting, and a parallel prefix-sum example. The dataset contains synthetic profiles and the website currently uses a separate local copy of the data, so it does not communicate with the Java application through an API. The system is intended for academic demonstration and not for making real hiring decisions.

# TABLE OF CONTENTS


| Section | Page |
|---|---|
| Declaration | i |
| Certificate | ii |
| Acknowledgement | iii |
| Abstract | iv |
| List of Figures | v |
| 1. Introduction | 1 |
| 1.1 Background and Overview | 1 |
| 1.2 Problem Statement | 1 |
| 1.3 Objectives and Scope | 2 |
| 2. System Requirements | 3 |
| 2.1 Existing Approach | 3 |
| 2.2 Functional Requirements | 3 |
| 2.3 Non-Functional Requirements | 4 |
| 2.4 Tools and Environment | 4 |
| 3. System Design | 5 |
| 3.1 Architecture | 5 |
| 3.2 Dataset and Data Representation | 6 |
| 3.3 Ranking Model | 6 |
| 3.4 Data Structures and Algorithms | 7 |
| 4. Implementation | 8 |
| 4.1 Module Description | 8 |
| 4.2 Search and Ranking Pseudocode | 9 |
| 4.3 Website Implementation | 10 |
| 4.4 Advanced DSA Demonstrations | 10 |
| 5. Results | 11 |
| 5.1 Functional Checks | 11 |
| 5.2 Demonstration Procedure | 12 |
| 6. Conclusion and Future Work | 13 |
| 6.1 Conclusion | 13 |
| 6.2 Limitations and Future Work | 14 |
| References | 15 |

# LIST OF FIGURES

3.1 High-level architecture of the Resume Search and Candidate Screening System

3.2 Candidate screening workflow

3.3 Weighted candidate fit score

5.1 TalentLens candidate search dashboard

5.2 Resume database and matching insights

# 1. INTRODUCTION

## 1.1 Background and Overview

Recruitment teams often need to review a large number of resumes and compare applicants against a role's requirements. Skills, years of experience, and location are common factors used during an initial review. When this work is performed manually, a reviewer may need to open each profile and repeatedly check the same fields. A structured search tool can make this first pass easier to repeat and explain.

The Resume Search and Candidate Screening System, presented through the TalentLens interface, is a local prototype for this task. A user enters required skills, a minimum experience value, and a preferred city. The system filters profiles by minimum experience and calculates a fit score from the selected criteria. The website presents the results as ranked candidate cards and also provides a database view and summary insights.

## 1.2 Problem Statement

The project addresses the problem of searching and comparing candidate profiles without manually inspecting every record. The system needs to store candidate information in a consistent structure, support search over relevant fields, rank the matching profiles using defined weights, and present the results clearly. The implementation also needs to demonstrate data structures and algorithms that are relevant to the course.

## 1.3 Objectives and Scope

- Load and browse candidate records from a local text dataset.
- Search using required skills, minimum experience, and preferred location.
- Rank candidates using a visible and repeatable weighted scoring model.
- Separate Java classes by responsibility so the implementation is easier to maintain.
- Provide a static website with Candidate Search, Resume Database, and Matching Insights views.
- Demonstrate selected string, dynamic-programming, graph, approximation, randomized, and parallel algorithms through small runnable examples.

The scope is limited to local demonstration. The included candidate profiles are synthetic. The website and Java console use separate data files and do not exchange data through a backend API.

# 2. SYSTEM REQUIREMENTS

## 2.1 Existing Approach and Limitations

In a manual screening workflow, a reviewer reads a resume, checks the required skills and experience, and decides whether the profile should be reviewed further. This can work for a small number of applicants but becomes repetitive as the number of records increases. A spreadsheet can organize candidate details, although consistent ranking requires carefully maintained filters or formulas.

The proposed prototype organizes selected fields and applies the same search and scoring rules to each record. It does not parse original PDF or Word resumes, connect to a recruitment database, or replace a production applicant-tracking system.

## 2.2 Functional Requirements

- Read candidate records from data/resumes.txt in the Java application.
- Create candidate objects and build a skill-to-candidate index.
- Accept skills, minimum experience, and preferred location as search criteria.
- Calculate weighted fit scores and order results from highest to lowest.
- Display candidate summaries and detailed profile information in the website.
- Search the resume database by candidate name, skill, city, or education.
- Display experience distribution, common skills, and algorithm summaries.
- Run selected algorithm examples from the Java console menu.

## 2.3 Non-Functional Requirements

- Usability: navigation and search fields should be clear during a demonstration.
- Maintainability: repository, ranking, algorithm, and interface code should be kept in separate files.
- Portability: the website should run in a modern browser; the console requires a compatible Java Development Kit.
- Transparency: the scoring weights and profile fields should be visible.
- Privacy: included profiles are synthetic and should not be interpreted as real applicant data.
- Reliability: the local dataset path and the Java compilation process should be handled consistently.

## 2.4 Tools and Environment

| Component | Technology / requirement |
|---|---|
| Backend language | Java |
| Website | HTML5, CSS3, JavaScript |
| Data files | TXT dataset for Java; JavaScript data file for the website |
| Development environment | Visual Studio Code or another code editor |
| Runtime | JDK with javac/java; modern web browser |
| Operating system | Windows run.bat; run.sh for compatible Unix-like environments |

# 3. SYSTEM DESIGN

## 3.1 Architecture

The implementation is organized into four logical parts: data, screening logic, algorithm demonstrations, and presentation. The Java console loads the TXT file and calls the repository, screening service, and ranking classes. The website loads a JavaScript data file and performs its own filtering and ranking in the browser. These are two independent local interfaces in the current version.

| Layer | Files / modules | Purpose |
|---|---|---|
| Data | data/resumes.txt; web/candidates.js | Stores synthetic candidate records for the two local interfaces. |
| Repository | Candidate.java; ResumeRepository.java | Represents profiles, loads records, and maintains the skill index. |
| Screening | ScreeningService.java; CandidateRanker.java | Processes criteria, calculates fit scores, and sorts results. |
| Algorithms | StringAlgorithms.java; AdvancedAlgorithms.java; TextUtils.java | Provides string, DP, graph, approximation, randomized, and parallel examples. |
| Presentation | ResumeScreeningSystem.java; web/index.html; styles.css; app.js | Provides the terminal menu and browser interface. |

*Fig. 3.1 High-level architecture of the Resume Search and Candidate Screening System.*

## 3.2 Dataset and Data Representation

The supplied dataset contains 1,000 synthetic candidate profiles. A record includes an ID, name, placeholder email address, education, years of experience, location, a list of skills, and a short summary. In the Java version, ResumeRepository reads the delimited text file and creates Candidate objects. It also builds a hash-based index from each skill to the IDs of candidates who list that skill.

The static website loads web/candidates.js, which contains a browser-side copy of the candidate records. This allows the page to run by opening web/index.html without installing npm packages or starting a server. Since the two interfaces use separate data copies, changes made to the TXT file are not automatically reflected in the website.

## 3.3 Ranking Model

The model calculates a score from skills, experience, and location. The weights are fixed in the current implementation and are displayed in the interface so the user can understand the main factors behind the ranking.

| Factor | Weight | Scoring approach |
|---|---|---|
| Skills match | 50% | The percentage of required skills matched against a candidate's listed skills. |
| Experience | 30% | Years of experience scaled against the dataset's 12-year range, capped at 100. |

| | | |
|---|---|---|
| Location | 20% | A preferred-city match receives the full location score; a different city receives zero for this component. |

**Fit Score = (0.50 × Skills Score) + (0.30 × Experience Score) + (0.20 × Location Score).**

Minimum experience is used as a filter. Preferred location affects the score rather than excluding every candidate from another city. When no skills are entered, the interface can show profiles ranked using the remaining criteria. The fit score is a ranking aid, not a probability of job performance.

## 3.4 Data Structures and Algorithms

| Concept | Use in the project |
|---|---|
| ArrayList / collections | Stores candidate objects and supports iteration over profiles. |
| HashMap and sets | Maintain a skill index and store unique normalized skills. |
| Sorting | Orders candidates by fit score with tie-breakers. |
| KMP and edit distance | Provide string matching and typo-comparison functionality/demonstrations. |
| Z-function, Rabin–Karp, suffix array | Additional string-processing implementations in AdvancedAlgorithms. |
| Dynamic programming | LCS, matrix-chain multiplication, edit distance, and bitmask TSP examples. |
| Maximum flow | Edmonds–Karp demonstrates augmenting paths and residual capacities. |
| Approximation | Vertex Cover approximation illustrates a simple approximation strategy. |
| Randomized and parallel algorithms | Randomized QuickSort and a parallel prefix-sum example. |

Not every algorithm is used in the live candidate-ranking path. The advanced examples are separate runnable demonstrations intended to support discussion of the course topics.

# 4. IMPLEMENTATION

## 4.1 Module Description

| File / module | Responsibility |
|---|---|
| Candidate.java | Stores candidate attributes and exposes the profile fields. |
| ResumeRepository.java | Loads the TXT dataset and builds candidate objects and the skill index. |
| ScreeningService.java | Processes search inputs and prepares normalized required skills. |
| CandidateRanker.java | Calculates the weighted fit score and orders candidates. |
| StringAlgorithms.java | Implements KMP matching and edit distance. |
| AdvancedAlgorithms.java | Contains the additional string, DP, maximum-flow, approximation, randomized, and parallel examples. |
| TextUtils.java | Normalizes text for comparisons. |
| ResumeScreeningSystem.java | Provides the Java console menu and prints results. |
| web/index.html | Defines the website's layout and main views. |
| web/styles.css | Defines typography, layout, colours, cards, and responsive styling. |
| web/app.js | Handles browser search, ranking, navigation, pagination, charts, and profile modals. |
| data/resumes.txt | Stores synthetic profiles for the Java backend. |

## 4.2 Search and Ranking Pseudocode

```
INPUT requiredSkills, minimumExperience, preferredLocation
LOAD candidate records
FOR each candidate:
    IF experience < minimumExperience:
        CONTINUE
    matchedSkills = compare required skills with listed skills
    skillsScore = matchedSkills / requiredSkills count * 100
    experienceScore = MIN(experience / 12 * 100, 100)
    locationScore = 100 if preferred city matches, otherwise 0
    fitScore = 0.50 * skillsScore
             + 0.30 * experienceScore
             + 0.20 * locationScore
    ADD candidate and fitScore to results
SORT results by fitScore descending
DISPLAY ranked results
```

The Java and browser implementations apply the same weight values but calculate scores independently. The website does not call the Java application through an API in this version.

## 4.3 Website Implementation

The website has three main views. Candidate Search accepts skills, a preferred location, and minimum experience, then displays ranked candidate cards. Resume Database provides text search and pagination over the candidate pool. Matching Insights shows the distribution of experience, the most common skills, and an explanation of the algorithm toolkit. Selecting a candidate opens a profile window containing the stored fields and skills.

The website is static and can be opened by launching web/index.html in a modern browser. It loads the local CSS and JavaScript files and does not require npm or a database server. The Java console is launched separately with run.bat.

## 4.4 Advanced DSA Demonstrations

The console menu includes an advanced-algorithm option with small examples of the Z-function, Rabin–Karp search, suffix-array construction, longest common subsequence, matrix-chain multiplication, bitmask TSP, Edmonds–Karp maximum flow, approximate Vertex Cover, randomized QuickSort, and parallel prefix sum. Small inputs make the output easier to follow during a demonstration.

These demonstrations should be described according to what the code actually implements. For example, the Vertex Cover routine is an approximation example and not an exact optimizer, while the suffix-array routine is a straightforward baseline construction rather than an optimized suffix-array algorithm.

# 5. RESULTS

## 5.1 Functional Checks

The project source was checked for Java compilation, frontend JavaScript syntax, and a console smoke test that executes the advanced DSA menu. The dataset contains 1,000 synthetic profiles. These checks establish that the source compiles and that the console examples can be run; they are not a formal performance benchmark.

| Check | Expected result | Outcome |
|---|---|---|
| Java compilation | Java source files compile together | Passed |
| Frontend JavaScript syntax | app.js parses without syntax errors | Passed |
| Advanced DSA menu | Example algorithms print output in the console | Passed |
| Website opening | index.html loads local CSS and JavaScript in a browser | Verify on presentation laptop |
| Search behaviour | Results reflect the selected filters and scoring weights | Demonstrate with chosen input |

## 5.2 Demonstration Procedure

For the website demonstration, open web/index.html in Chrome. On Candidate Search, enter skills such as “java, sql”, set minimum experience to 1 year, and choose Any location. Click Find matching candidates and inspect the ranked profiles. Then open Resume Database to demonstrate searching the dataset and Matching Insights to explain the scoring weights and algorithm examples.

For the Java console demonstration, run .\run.bat from the project root. Choose option 1 to search and rank candidates, option 2 to browse the dataset, option 3 to view the skill index, option 4 to show the scoring model, and option 5 to run the advanced DSA examples.

*Fig. 5.1 TalentLens candidate search dashboard — insert an actual screenshot of the running website.*

*Fig. 5.2 Resume database and matching insights — insert actual screenshots of the running website.*

*Screenshots are intentionally marked as insertion points rather than fabricated. Add the screenshots from the exact project version used during review.*

# 6. CONCLUSION AND FUTURE WORK

## 6.1 Conclusion

The Resume Search and Candidate Screening System provides a local way to search candidate profiles and rank them using skills, experience, and location. The Java implementation separates candidate data, repository loading, screening, ranking, and algorithm examples into different files. The website presents the same type of workflow through a dashboard with candidate search, database browsing, and matching insights.

The project demonstrates how collections, indexing, sorting, string processing, dynamic programming, graph algorithms, approximation, and randomized/parallel techniques can be discussed through a practical example. The scoring model is visible and repeatable, but the outcome depends on the selected criteria and fixed weights. Since the records are synthetic and the website is not connected to the Java process, the project remains an academic prototype.

## 6.2 Limitations and Future Work

The current system has several limitations. The profiles are synthetic, the scoring weights are fixed, and simple text-matching rules can produce false matches. The experience score is scaled against a 12-year range, which may not suit every role. The website and Java console use separate data files, and the application does not parse PDF or Word resumes, authenticate users, maintain audit logs, or integrate with an applicant-tracking system.

Future work could connect the website to the Java screening service through an API, use a single source of candidate data, add controlled skill synonyms, write unit tests for the ranking and algorithm modules, and allow scoring weights to be configured. The project could also add resume parsing and role-specific screening criteria after the current search behaviour has been tested carefully.

# REFERENCES

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. MIT Press, 2022.

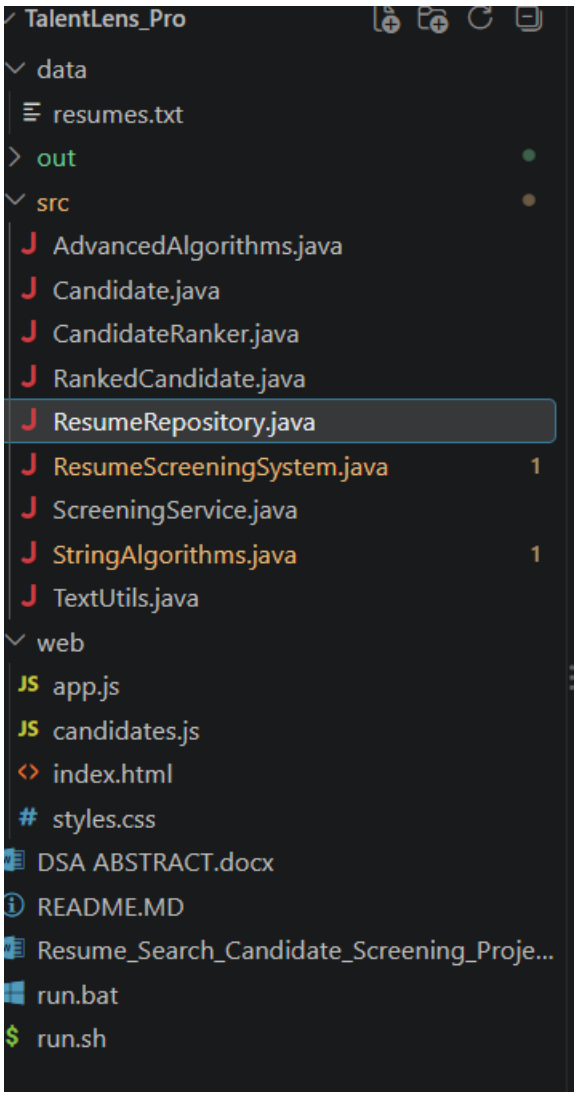
[2] R. Sedgewick and K. Wayne, Algorithms, 4th ed. Addison-Wesley, 2011.

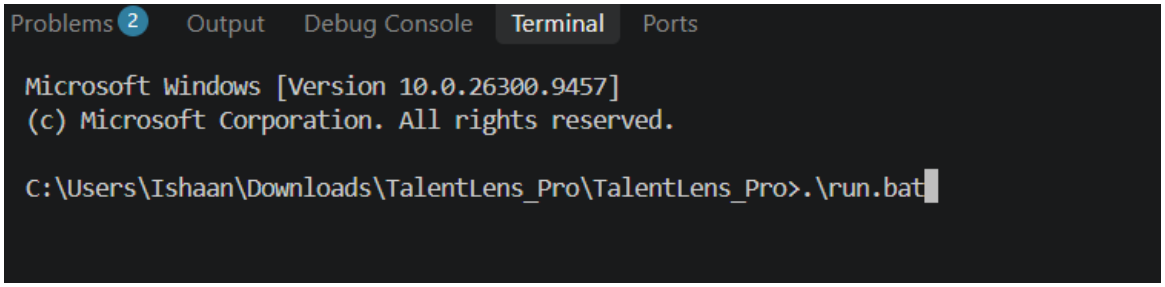
[3] Oracle, “Java Documentation.” Available: https://docs.oracle.com/en/java/

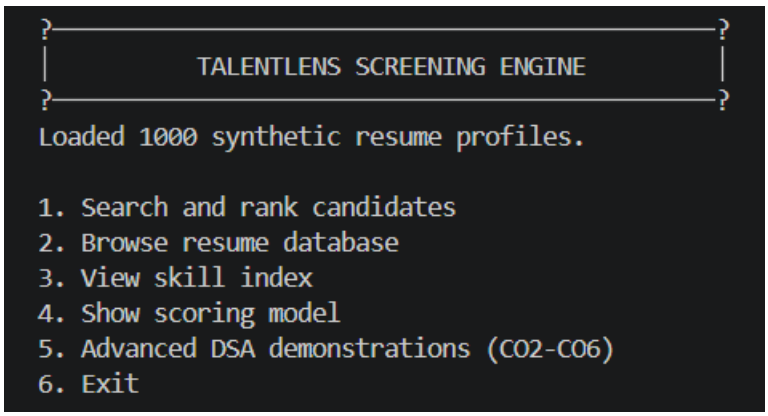
[4] MDN Web Docs, “HTML, CSS, and JavaScript.” Available: https://developer.mozilla.org/

[5] Department of Computer Science and Engineering, Data Structures and Algorithms–3 (25CS2103E), course handout, academic year 2026–27.

# SCREENSHOTS



```
Choose an option: 3
algorithms              274 candidates
aws                     279 candidates
c++                     251 candidates
computer vision         273 candidates
css                     241 candidates
data analysis           332 candidates
data structures         259 candidates
database                236 candidates
deep learning           264 candidates
```

```
Choose an option: 2
  1 | Zoya Bose              |  6 yrs | Mumbai         | css, database, html, node.js, postgresql, python,
spring boot
  2 | Aditi Khan             | 12 yrs | Jaipur         | data structures, html, mongodb, power bi, python,
scikit-learn, tableau
  3 | Sara Reddy             |  5 yrs | Chennai        | c++, data analysis, git, javascript, node.js, spri
ng boot, tableau, tensorflow
  4 | Diya Mehta             |  3 yrs | Bengaluru      | computer vision, docker, flask, mongodb, mysql, nu
mpy
```

```
1. Search and rank candidates
2. Browse resume database
3. View skill index
4. Show scoring model
5. Advanced DSA demonstrations (CO2-CO6)
6. Exit
Choose an option: 4
Skills match: 50% | Experience: 30% | Location: 20%
Experience score scales from 0 to 12 years; location is a preferred-city match.
```

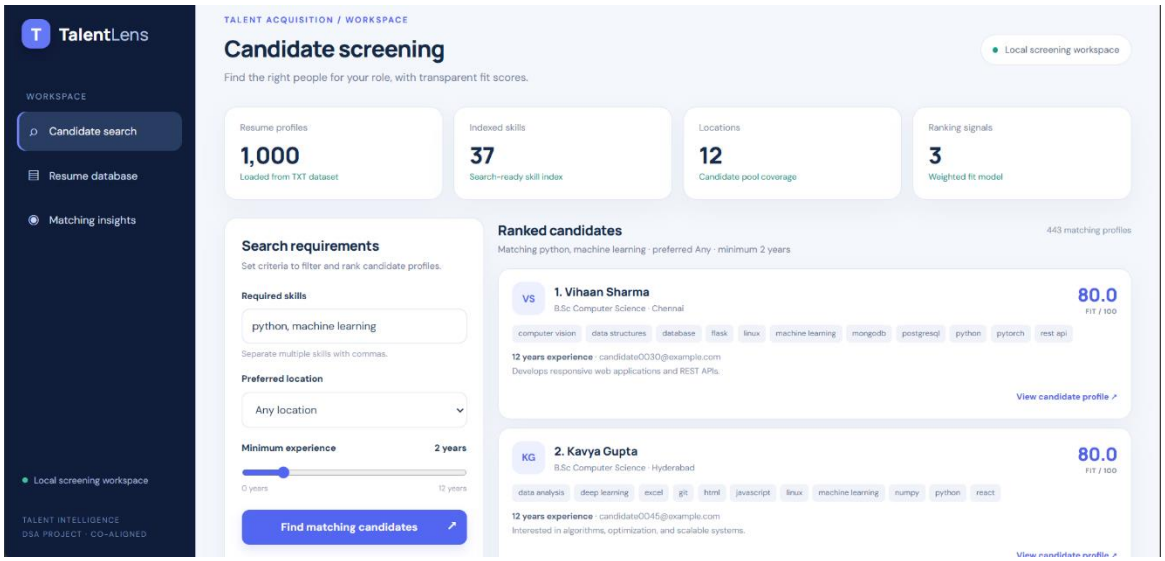
src > StringAlgorithms.java > ...

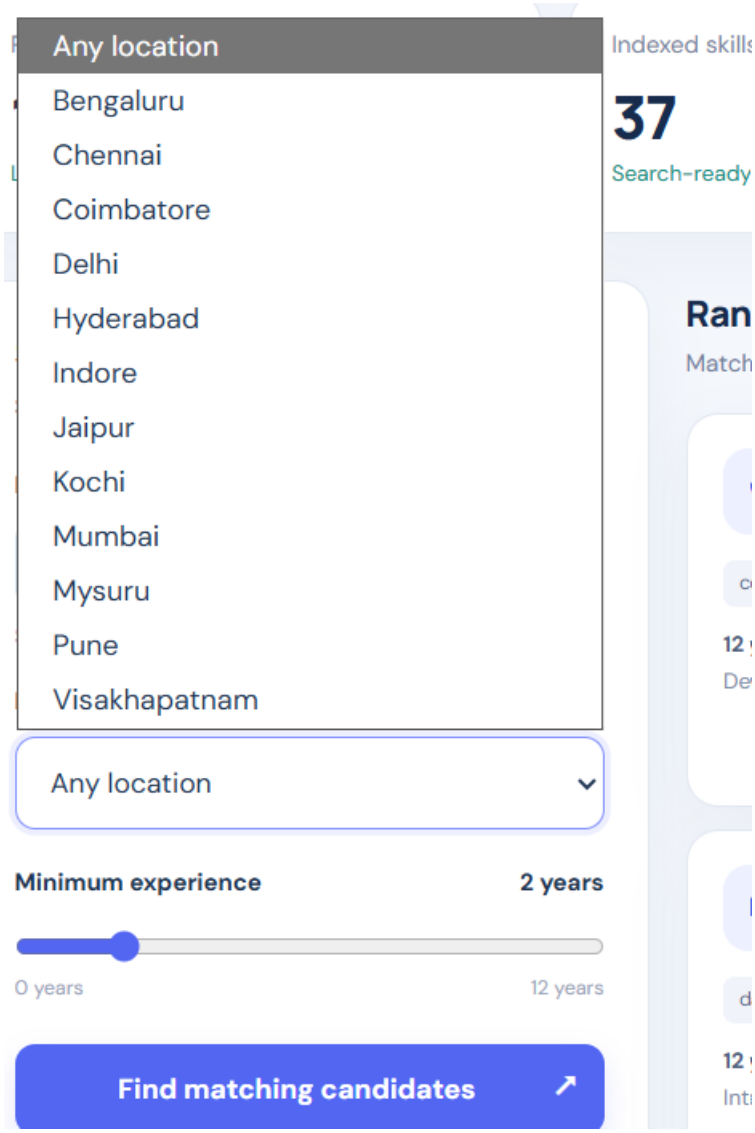
```java
import java.util.*;

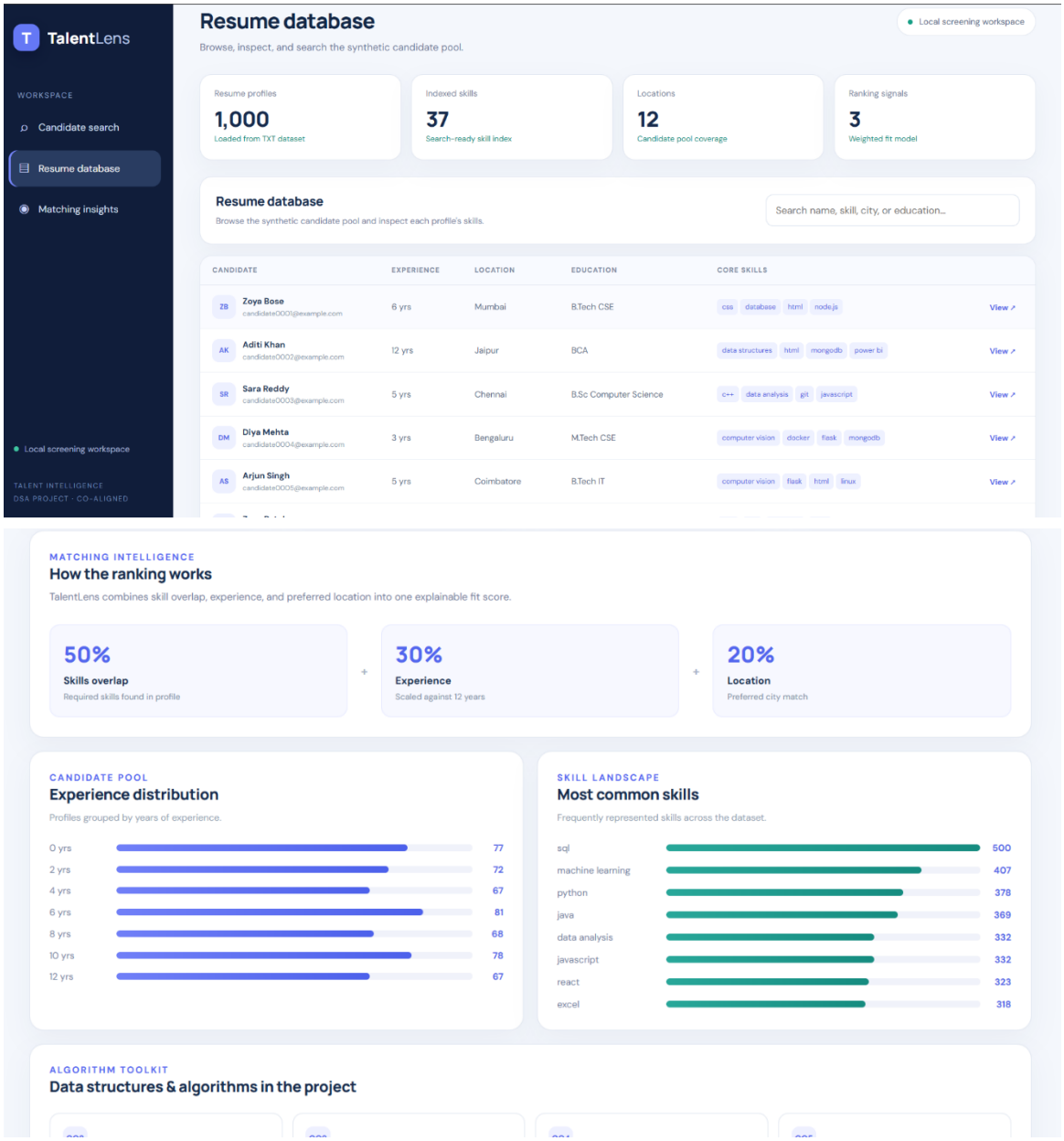
public final class StringAlgorithms {
    private StringAlgorithms() {}

    public static boolean kmpContains(String text, String pattern) {
        text = TextUtils.normalize(text); pattern = TextUtils.normalize(pattern);
        if (pattern.isEmpty()) return true;
        int[] lps = new int[pattern.length()];
        for (int i=1, len=0; i<pattern.length();) {
            if (pattern.charAt(i)==pattern.charAt(len)) lps[i++]=++len;
            else if (len>0) len=lps[len-1]; else lps[i++]=0;
        }
        for (int i=0,j=0; i<text.length();) {
            if (text.charAt(i)==pattern.charAt(j)) {
                i++; j++; if (j==pattern.length()) return true;
            } else if (j>0) j=lps[j-1]; else i++;
        }
        return false;
    }

    public static int editDistance(String a, String b) {
        a=TextUtils.normalize(a); b=TextUtils.normalize(b);
```


TalentLens
WORKSPACE
Candidate search
Resume database
Matching insights
Local screening workspace
TALENT INTELLIGENCE
DSA PROJECT · CO-ALIGNED
TALENT ACQUISITION / WORKSPACE
Candidate screening
Find the right people for your role, with transparent fit scores.
Local screening workspace
Resume profiles
1,000
Loaded from TXT dataset
Indexed skills
37
Search-ready skill index
Locations
12
Candidate pool coverage
Ranking signals
3
Weighted fit model
Search requirements
Set criteria to filter and rank candidate profiles.
Required skills
python, machine learning
Separate multiple skills with commas.
Preferred location
Any location
Minimum experience
2 years
0 years
12 years
Find matching candidates
Ranked candidates
443 matching profiles
Matching python, machine learning · preferred Any · minimum 2 years
VS
1. Vihaan Sharma
B.Sc Computer Science · Chennai
80.0
FIT / 100
computer vision
data structures
database
flask
linux
machine learning
mongodb
postgresql
python
pytorch
rest api
12 years experience · candidate0030@example.com
Develops responsive web applications and REST APIs.
View candidate profile
KG
2. Kavya Gupta
B.Sc Computer Science · Hyderabad
80.0
FIT / 100
data analysis
deep learning
excel
git
html
javascript
linux
machine learning
numpy
python
react
12 years experience · candidate0045@example.com
Interested in algorithms, optimization, and scalable systems.


Any location
Bengaluru
Chennai
Coimbatore
Delhi
Hyderabad
Indore
Jaipur
Kochi
Mumbai
Mysuru
Pune
Visakhapatnam
Any location
Minimum experience
2 years
0 years
12 years
Find matching candidates
Indexed skills
37
Search-ready


TalentLens
WORKSPACE
Candidate search
Resume database
Matching insights
Local screening workspace
TALENT INTELLIGENCE
DSA PROJECT · CO-ALIGNED
Resume database
Browse, inspect, and search the synthetic candidate pool.
Local screening workspace
Resume profiles
1,000
Loaded from TXT dataset
Indexed skills
37
Search-ready skill index
Locations
12
Candidate pool coverage
Ranking signals
3
Weighted fit model
Resume database
Browse the synthetic candidate pool and inspect each profile's skills.
Search name, skill, city, or education...
CANDIDATE
EXPERIENCE
LOCATION
EDUCATION
CORE SKILLS
Zoya Bose
6 yrs
Mumbai
B.Tech CSE
css
database
html
node.js
View
Aditi Khan
12 yrs
Jaipur
BCA
data structures
html
mongodb
power bi
View
Sara Reddy
5 yrs
Chennai
B.Sc Computer Science
c++
data analysis
git
javascript
View
Diya Mehta
3 yrs
Bengaluru
M.Tech CSE
computer vision
docker
flask
mongodb
View
Arjun Singh
5 yrs
Coimbatore
B.Tech IT
computer vision
flask
html
linux
View


MATCHING INTELLIGENCE
How the ranking works
TalentLens combines skill overlap, experience, and preferred location into one explainable fit score.
50%
Skills overlap
Required skills found in profile
30%
Experience
Scaled against 12 years
20%
Location
Preferred city match
CANDIDATE POOL
Experience distribution
Profiles grouped by years of experience.
0 yrs 77
2 yrs 72
4 yrs 67
6 yrs 81
8 yrs 68
10 yrs 78
12 yrs 67
SKILL LANDSCAPE
Most common skills
Frequently represented skills across the dataset.
sql 500
machine learning 407
python 378
java 369
data analysis 332
javascript 332
react 323
excel 318
ALGORITHM TOOLKIT
Data structures & algorithms in the project